



End-to-End Navigation II

Course AIAA 4220

Week 10- Lecture 20

Changhao Chen
Assistant Professor
HKUST (GZ)

End-to-End Self-Driving

2014–2018: Modular + HD map

2019–2021: Sensor fusion + planning

2022–2023: BEV + multi-task learning

2024→ : End-to-end Navigation + VLM grounding



End-to-End Self-Driving

Pipeline Overview:

Sensors → BEV → Perception → Behavior prediction → Trajectory → Control

Modules include:

- BEV mapping
- Object & lane detection
- Agent intent prediction
- Trajectory head
- Safety controller

Choice	Options
Input	Camera / LiDAR / IMU / GPS
Representation	Raw pixels / BEV / voxel grid / 4D attention
Output	Control / trajectory / affordances
Training	IL, RL, self-supervision
World Model	Transformer temporal memory, diffusion models

End-to-End Self-Driving

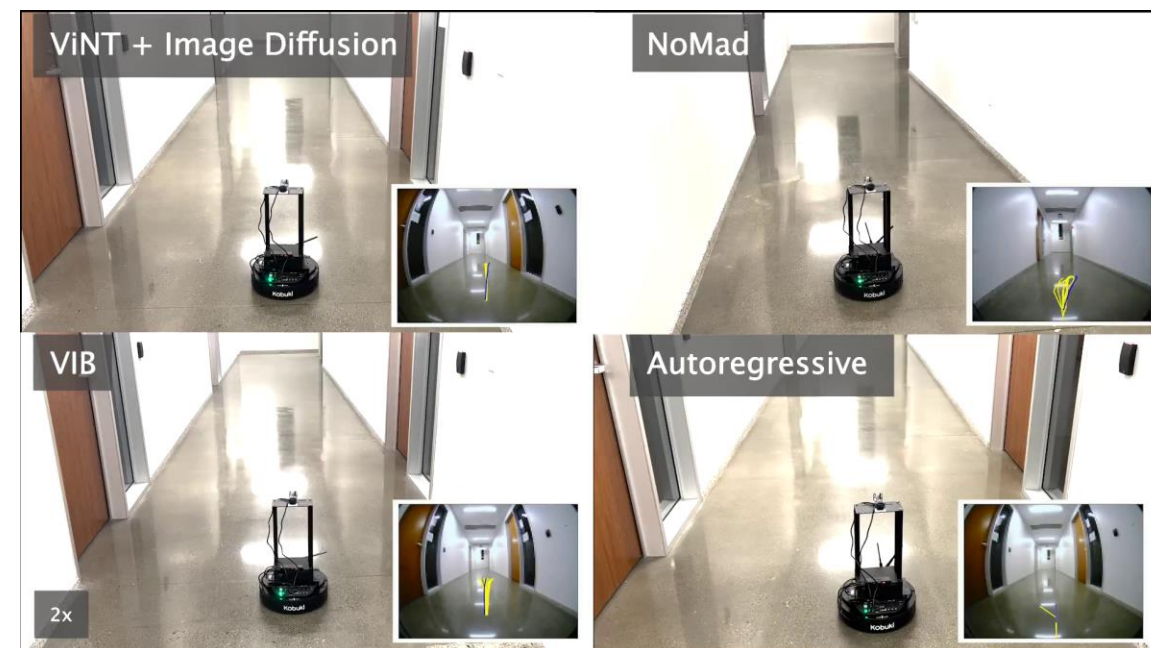
Why End-to-End for Driving?

Conventional stack issues:

- Manual feature engineering brittle
- HD maps expensive & degrade
- Strong human priors limit generalization

End-to-end advantages:

- Joint learning = fewer failure cascades
- Learning from real driving behavior
- Better long-tail behavior & emergent priors



Typical End-to-End Self-Driving Models

Direct Control Prediction

Core Concept: A single deep learning model maps raw sensor data (e.g., images) directly to vehicle control commands (steering, throttle, brake).

Exemplar Models: PilotNet (NVidia), CILRS (Wayve), LAV (UC Berkeley).

Key Advantage: Ultra-fast inference, enabling real-time reaction.

Primary Challenge: Limited interpretability, creating a "black box" problem for validation and debugging.

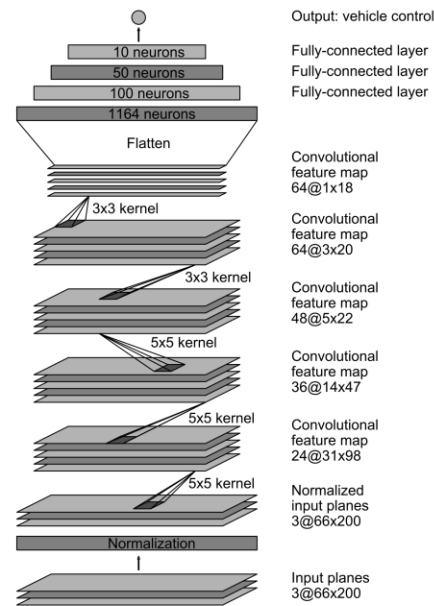
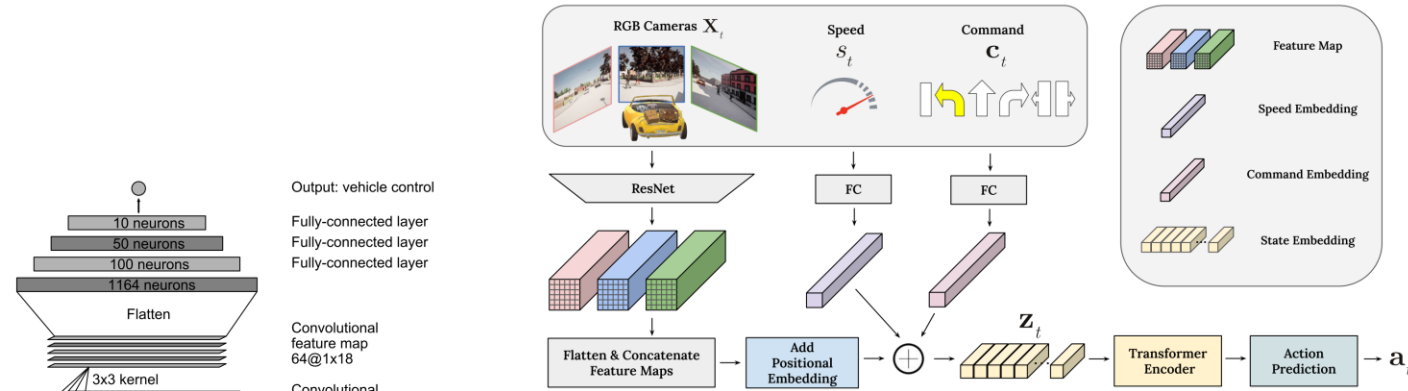
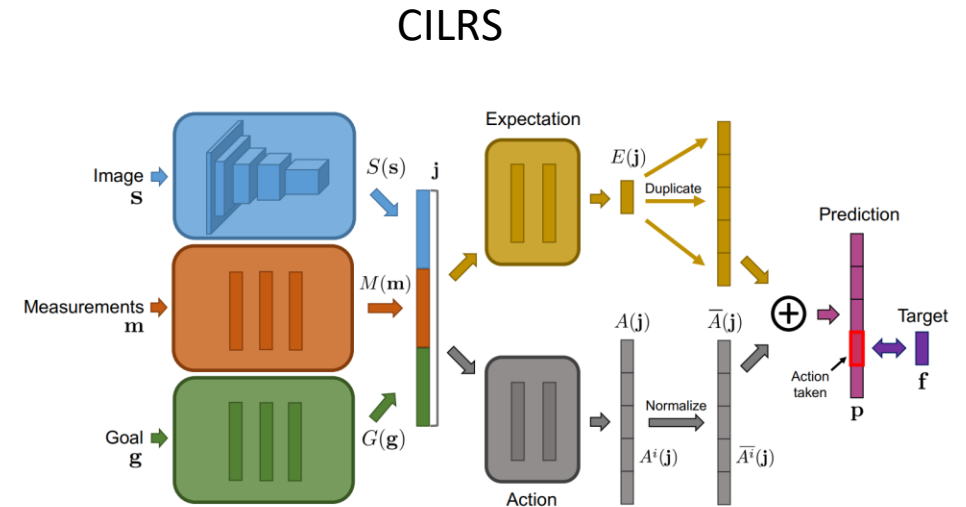


Figure 1: PilotNet architecture.

PilotNet



LAV

Typical End-to-End Self-Driving Models

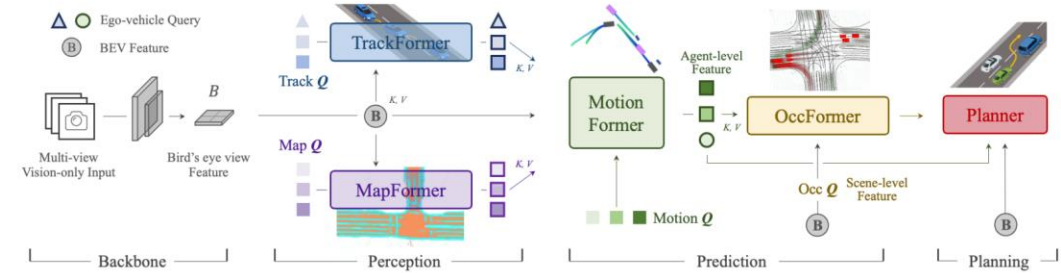
Intermediate Representation-Driven Models

Exemplar Models: UniAD, VAD, TCP

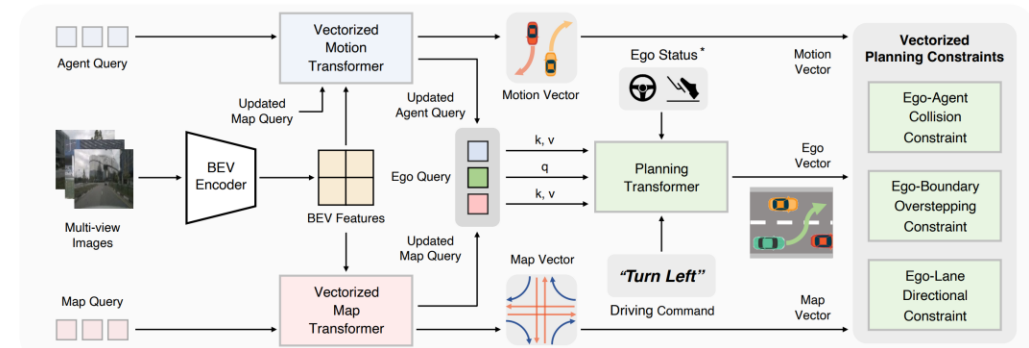
Core Innovation: Learns a rich, internal Bird's-Eye View (BEV) or semantic map as a foundational world model.

Structured Workflow:

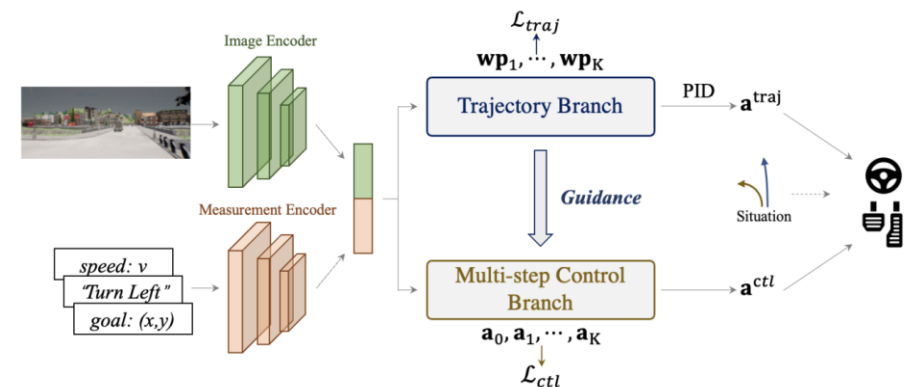
- Perception: Fuses sensor data to construct a dynamic BEV map, encoding the geometry and semantics of the scene.
- Planning: Uses this rich representation to predict others' trajectories and plan a safe, comfortable, and compliant ego-path.
- Control: Executes the planned trajectory with smooth and precise vehicle control.



UniAD



VAD

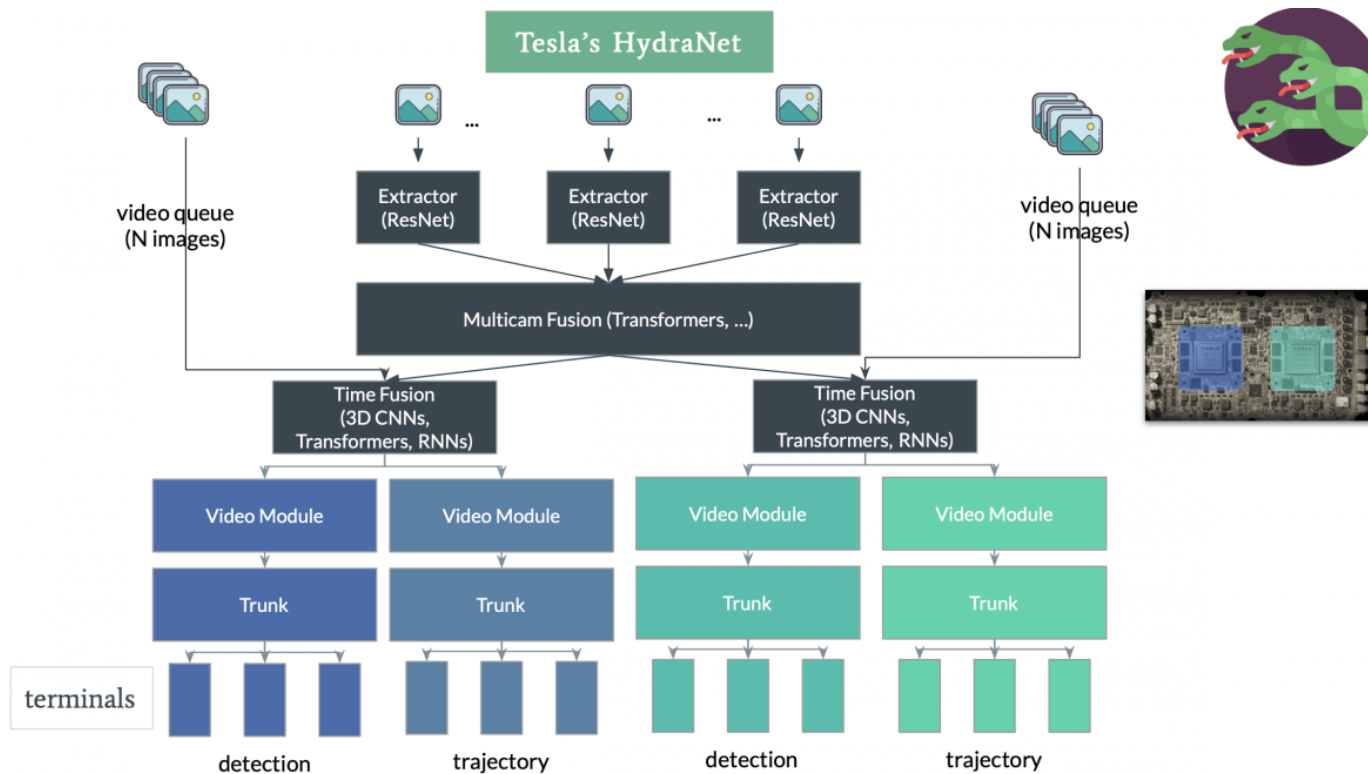


TCPNet

Case Study: Tesla FSD (Vision-only)

Tesla FSD: Fully end-to-end systems (2021):

"It's like Chat-GPT, but for cars!!!" "Instead of determining the proper path of the car based on rules, we determine the car's proper path by using neural network that learns from millions of training examples of what humans have done."

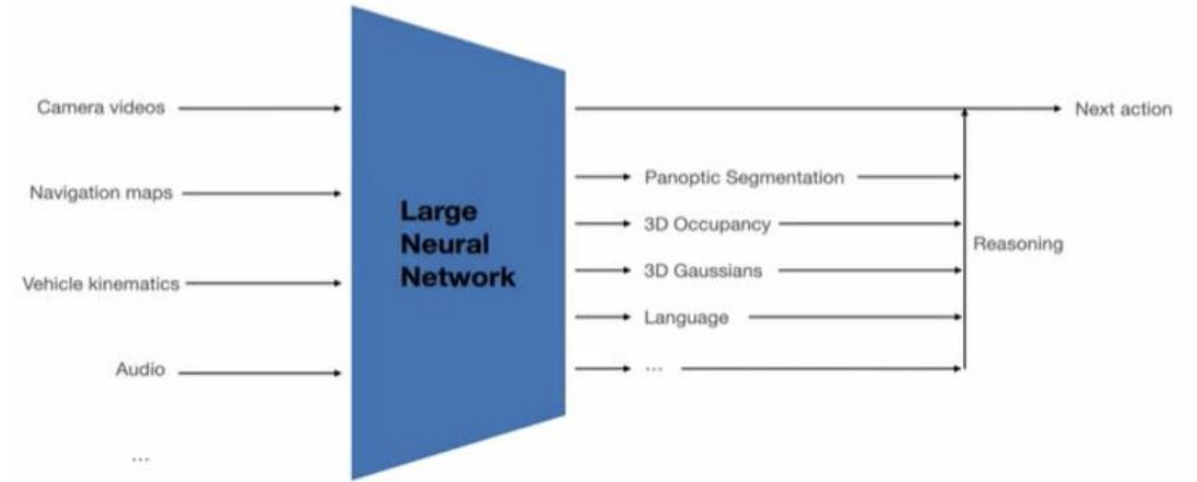


Case Study: Tesla FSD (Vision-only)

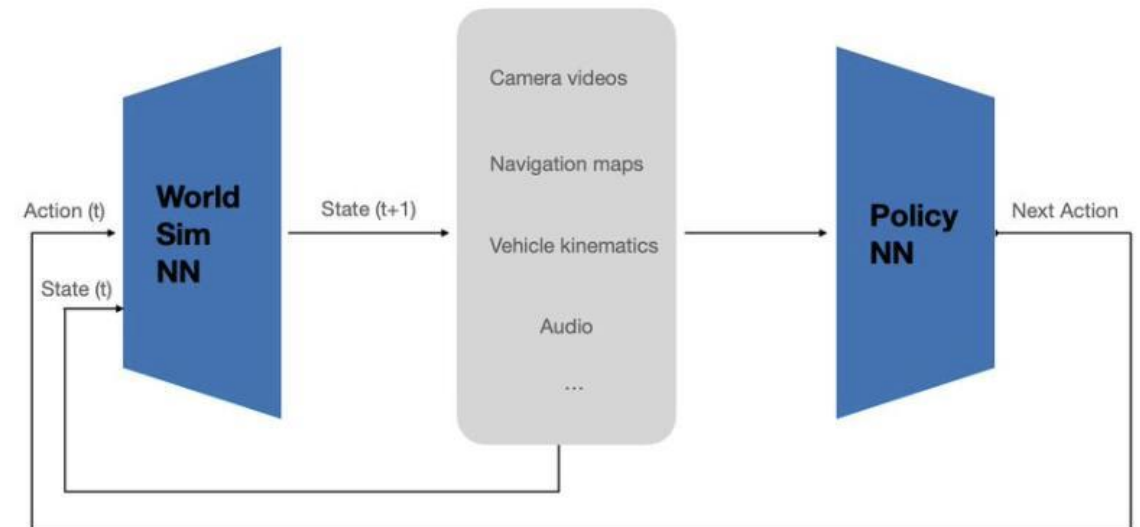
Tesla FSD: Fully end-to-end systems (2025)

Main Challenges of learning pixels -> control

1. Curse of dimensionality
2. Interpretability and safety guarantees
3. Evaluation



Interpretability and safety guarantees



Neural Network closed-loop simulator Can be trained with cheap to collect state-action data

Case Study: Wayve

Our innovative approach replaces the modular 'sense-plan-act' architecture of the traditional AV1.0 approach with a single neural network trained on diverse data to convert raw sensor inputs into safe driving outputs.

Driving Input, 10^8 dimensions



Cameras (6 @ 25 Hz)



GNSS



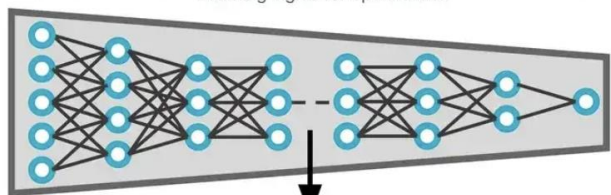
Basic Sat-nav Map



Vehicle State

+ other sensing modalities
where required, e.g. RADAR

Representation signal
Learning signal for optimisation



Driving Output, 10^1 dimensions

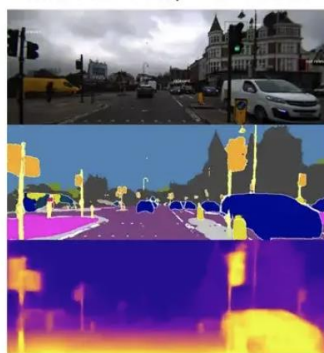


Motion Plan



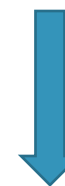
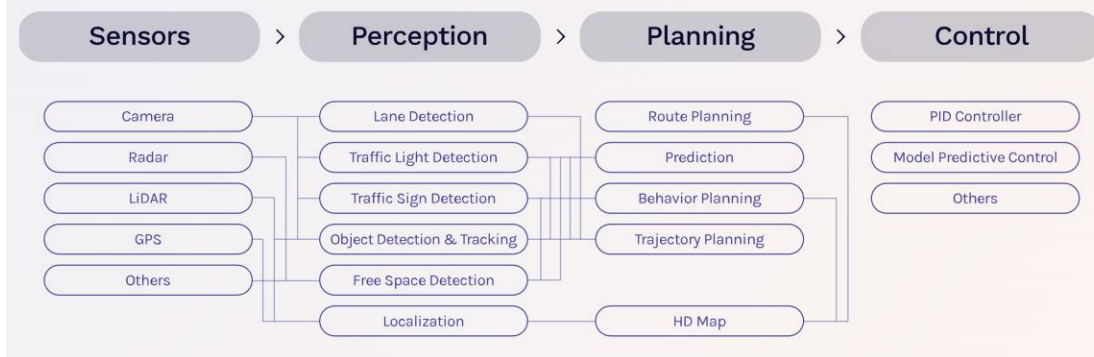
Vehicle Controls

Decoded human-interpretable
intermediate representations

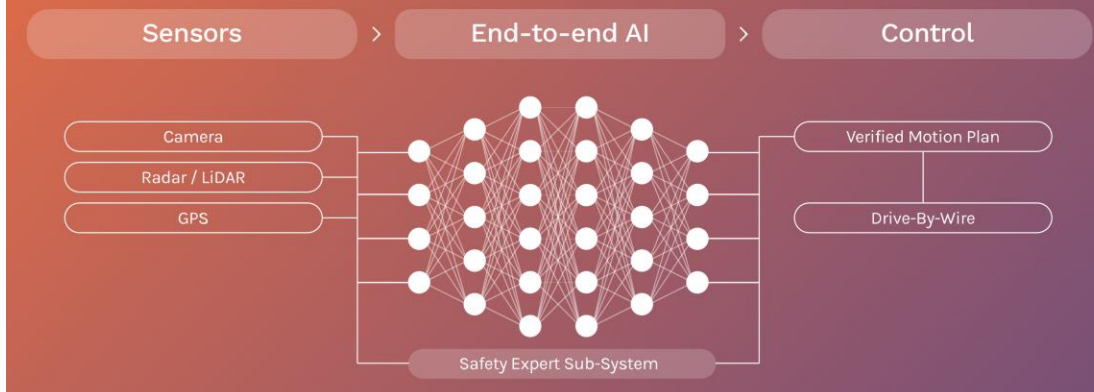


Semantics, geometry, motion prediction.

AV1.0



AV2.0



Safety & Reliability

Key challenges:

- Corner cases
- Uncertainty calibration
- Formal safety layers
- Rule grounding & driver norms
- Adversarial robustness
- Regulatory requirements

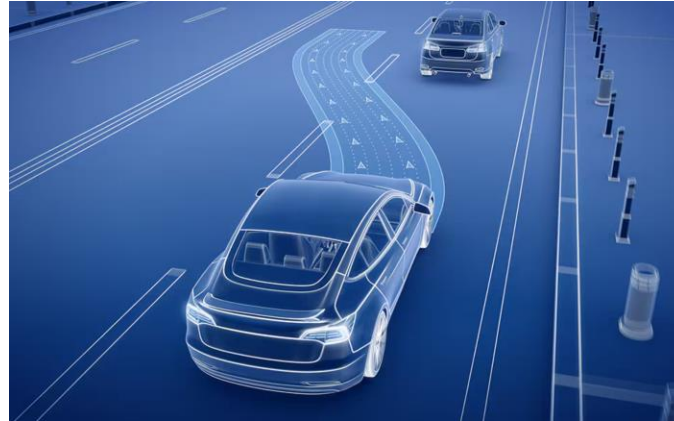
Takeaways :

- E2E does not mean “black-box”
- BEV + temporal transformers dominate
- Industry uses hybrid safety stack
- Learned planning → improved generalization

Embodied Navigation Beyond Cars

Platforms:

- Indoor service robots
- Warehouse AMRs
- Delivery robots (sidewalk bots)
- Drones (inspection, racing)
- Home humanoids



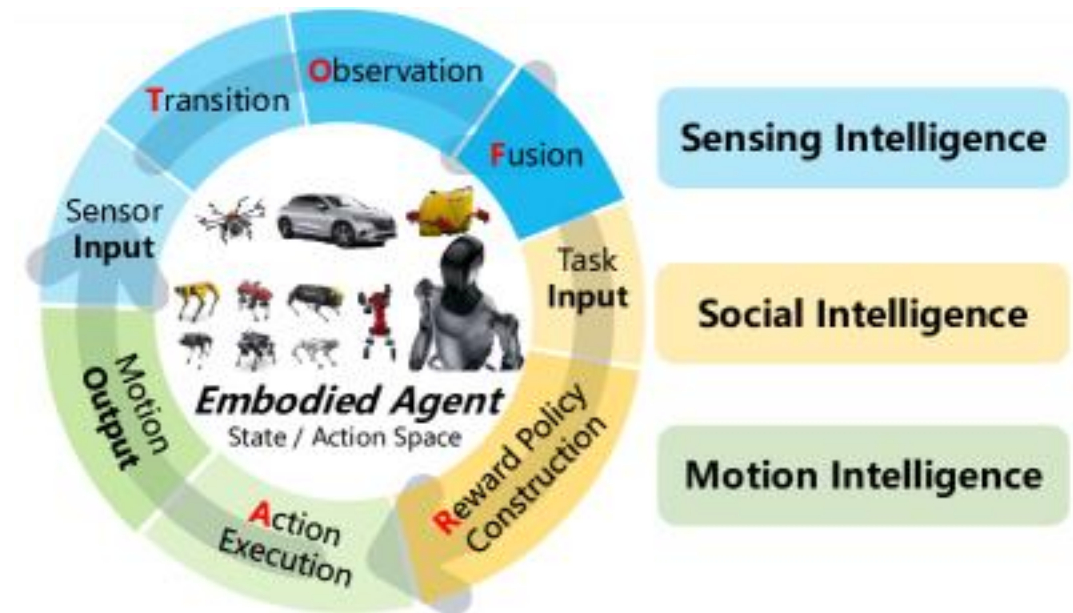
Embodied Navigation Loop

Pipeline:

Observe → Understand → Navigate → Interact

Sensors:

- RGB, depth, stereo
- LiDAR, IMU, wheel odometry
- Language instructions (VLM era)
- Tactile for manipulation robots



Source: Sensing, Social, and Motion Intelligence in Embodied Navigation: A Comprehensive Survey

End-to-End Robot Navigation

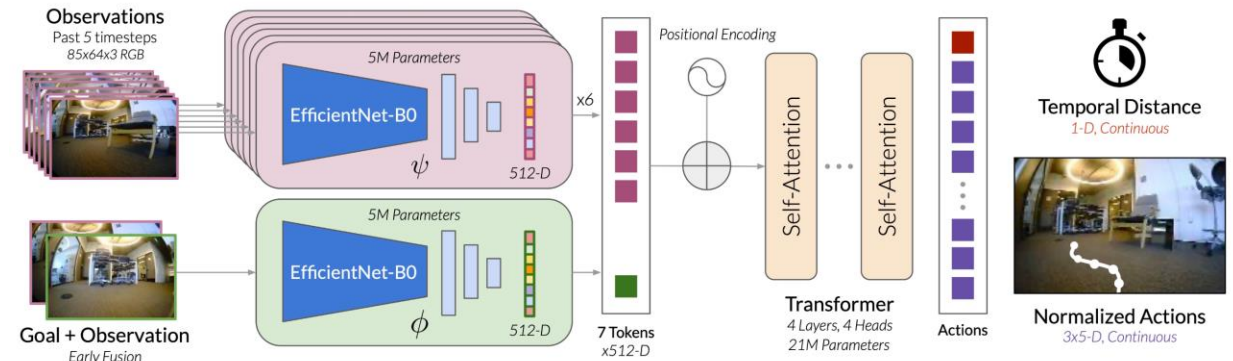
Vision based Robot Navigation Model:

ViNT: A foundation model for visual navigation

Architecture: Transformer-based, pre-trained on large-scale, heterogeneous robot data.

Mechanism: Goal-conditioned sequence prediction. Encodes a history of images and a goal image to autoregressively predict velocity commands.

Strength: Demonstrates strong few-shot adaptation and generalization to new robots and environments.



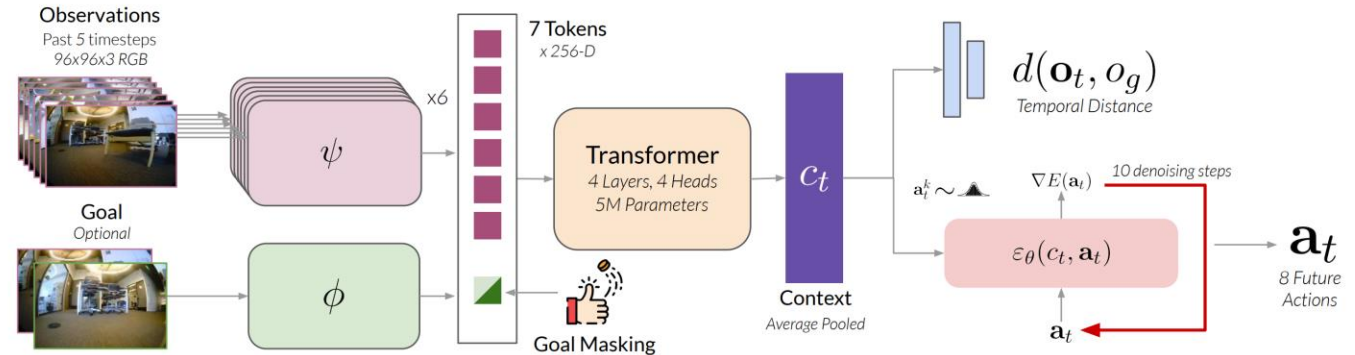
End-to-End Robot Navigation

Vision based Robot Navigation Model:

NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration

Imitation learning model that uses diffusion policies to generate robust and diverse navigation behaviors.

Core Idea: Employs a masked goal-conditioning strategy and normalized data to handle multi-modal scenarios and improve generalization.



End-to-End Robot Navigation

Vision-Language-Action (VLA) Agents

Agents that map:

(Images,Language)→Navigation+Manipulation

Examples:

- SayCan / RT-1 / RT-2
- PaLM-E
- MobileVLA / Embodied-GPT

End-to-End Robot Navigation

Vision-Language Navigation (VLN)

Task examples:

- “Go to the kitchen and find the refrigerator.”
- “Navigate to the red sofa and open the drawer.”

Benchmarks:

- R2R, RxR
- ALFRED / Ego-VQA
- Habitat-VLN
- BEHAVIOR-1K

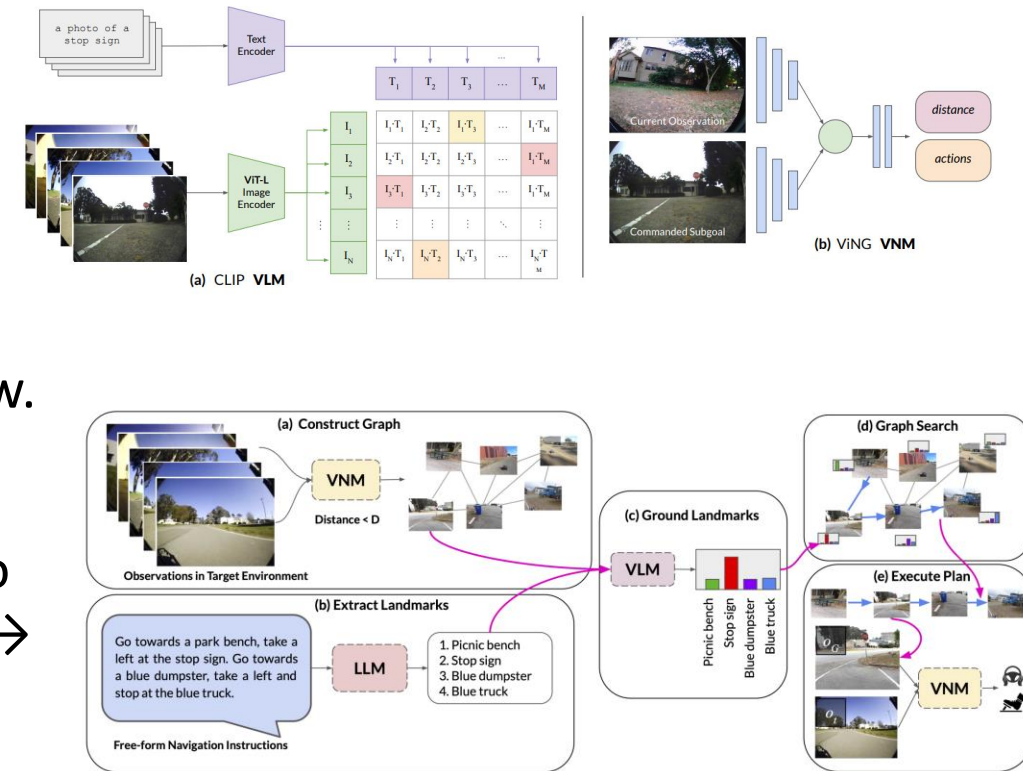
End-to-End Robot Navigation

Typical VLN Model:

LM-Nav:

Concept: Uses a large language model (LLM) as a semantic planner. It translates high-level natural language commands ("get my coffee from the kitchen") into a sequence of visual landmarks to follow.

Flow: Language $\rightarrow$ LLM $\rightarrow$ Landmark Sequence ("go to the hallway, then the kitchen") $\rightarrow$ Visuomotor Policy $\rightarrow$ Actions.



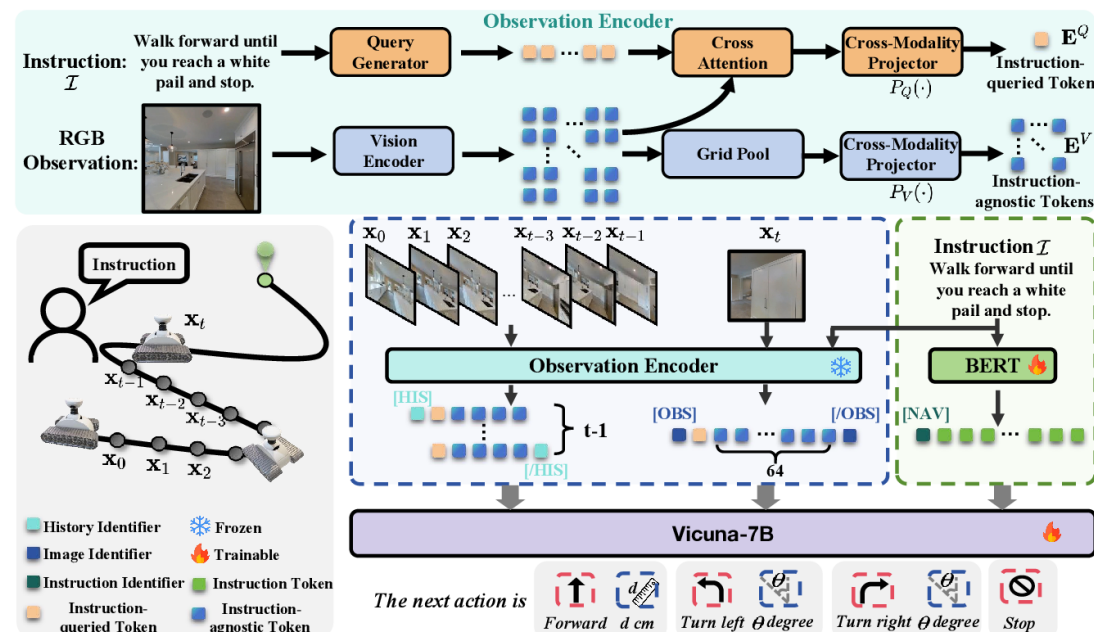
End-to-End Robot Navigation

Typical VLN Model:

Navid:

Concept: An LLM-powered multimodal agent that controls a robot directly from instructions and visual input. It reasons about actions in real-time.

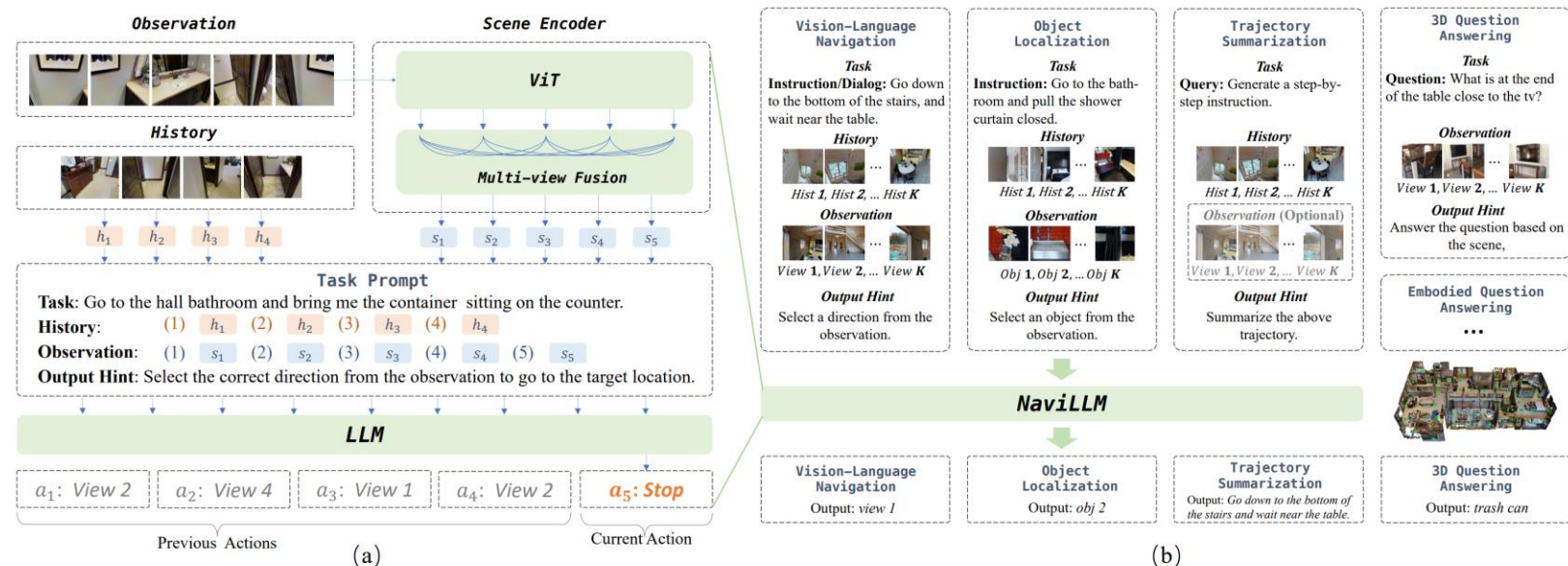
Flow: Continuously integrates language instruction, camera images, and its own actions to decide the next move, enabling closed-loop control.



End-to-End Robot Navigation

Typical VLN Model:

GPT-Navigator:
Extend Large Language Models (LLMs) from simple text processing to embodied AI applications that can understand and navigate the physical world.



Source: D. Zheng, S. Huang, L. Zhao, Y. Zhong, and L. Wang, "Towards learning a generalist model for embodied navigation," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 13 624–13 634.

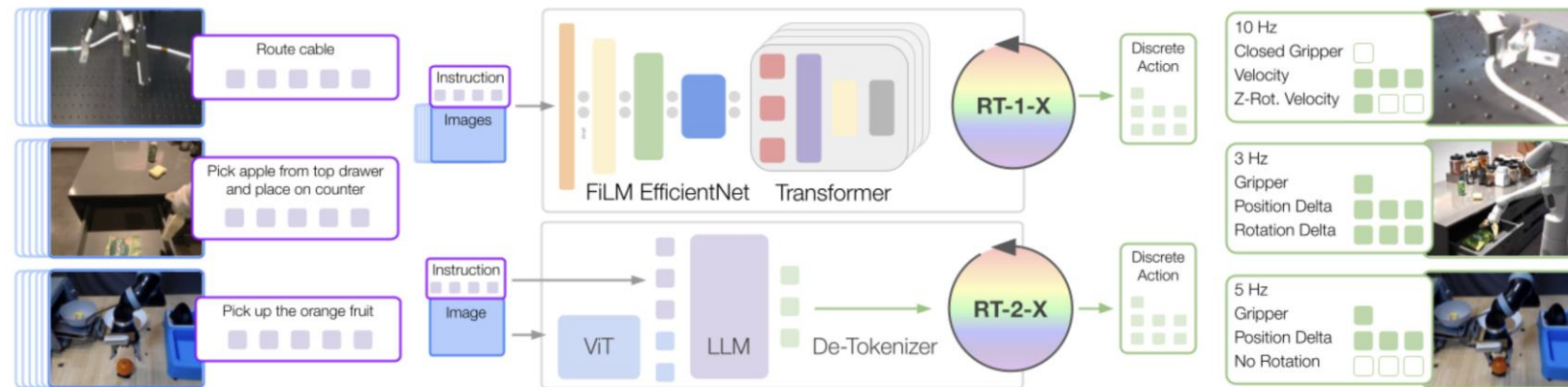
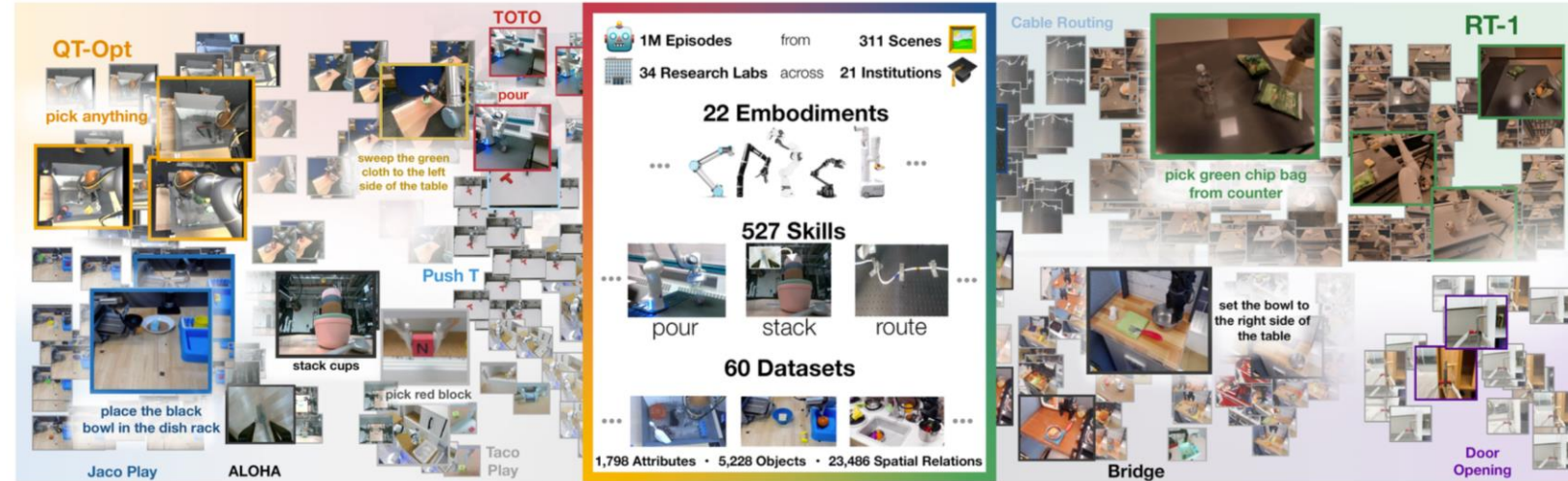
Generalist Robot Models

Trends:

- Single model → many robots/tasks
- Action tokens for robots
- Diffusion-based trajectory planners
- LLM as policy & skill routing

Examples:

- Open-X Embodiment
- RT-X



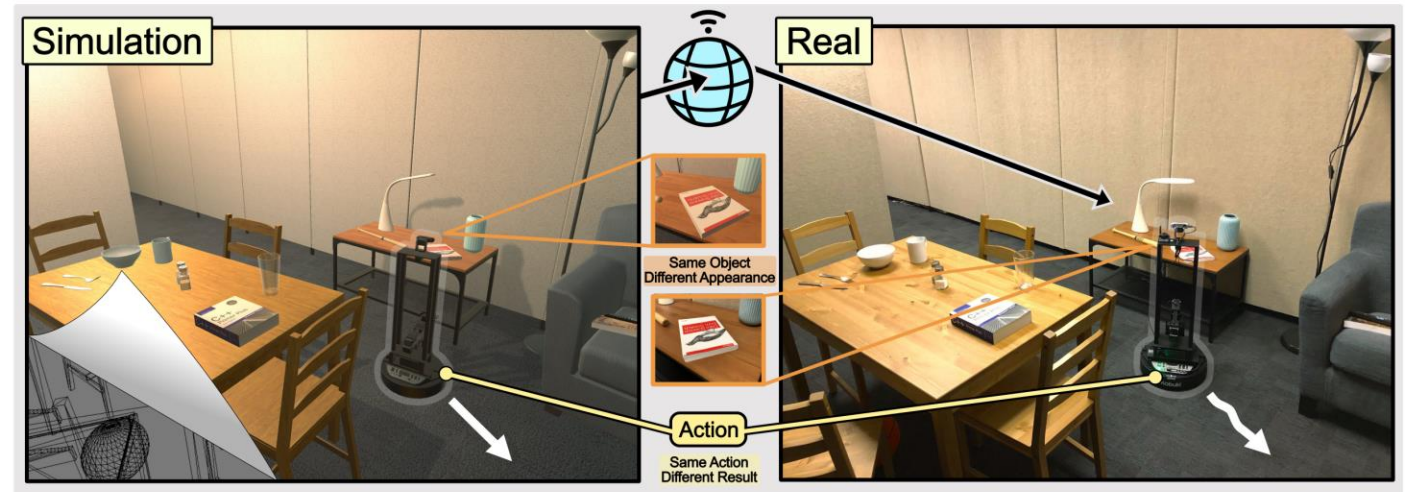
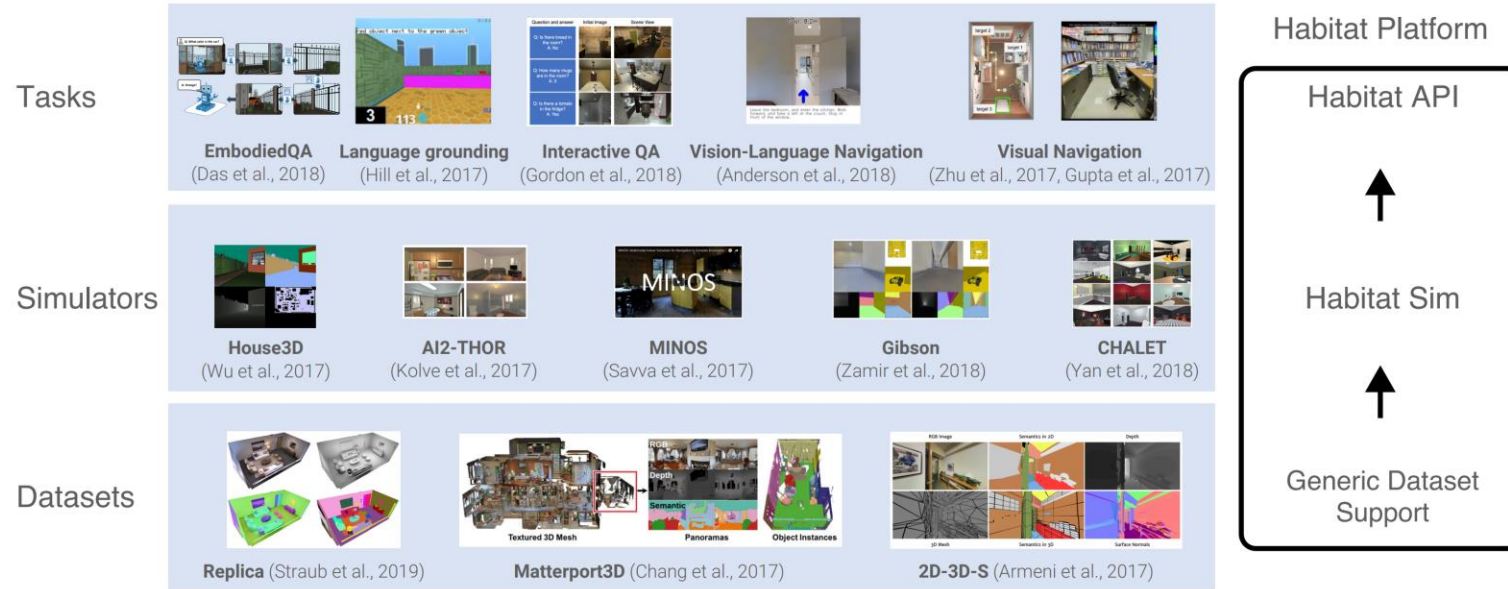
Simulation Platforms

Habitat:

A flexible, high-performance 3D simulator with configurable agents, multiple sensors, and generic 3D dataset handling

RoboTHOR:

RoboTHOR is an environment within the AI2-THOR framework, designed to develop embodied AI agents. It consists of a series of scenes in simulation with counterparts in the physical world.



Simulation Platforms

Isaac Sim:

NVIDIA Isaac Sim™ is an open-source reference application built on NVIDIA Omniverse™ that enables developers to simulate and test AI-driven robotics solutions in physically based virtual environments.



Learning Paradigms for Navigation Models

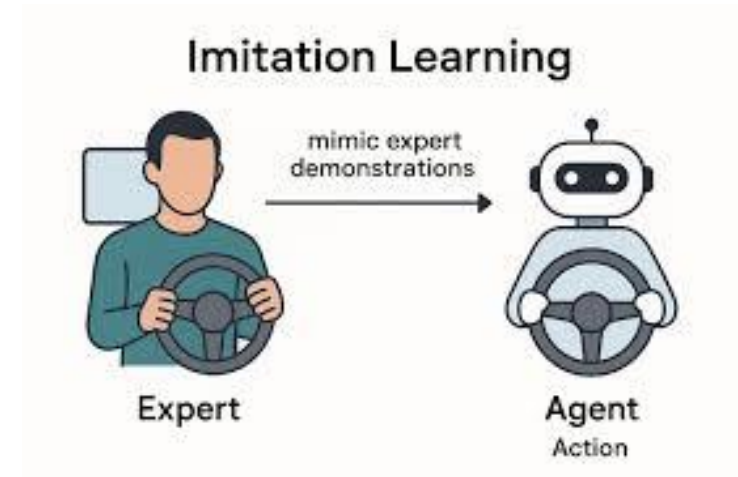
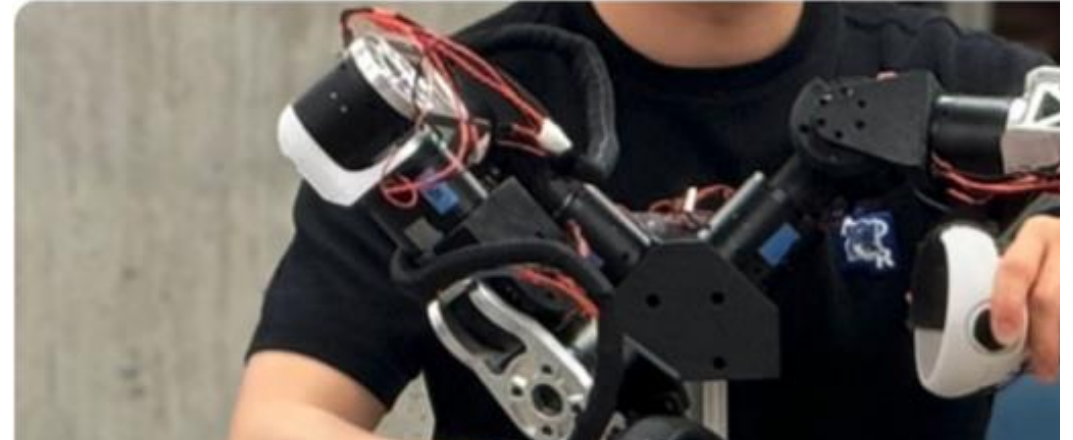
Imitation Learning

Behavioral Cloning (BC): Mimics an expert's state-action mapping via supervised learning.

Dataset Aggregation (DAGger): Iteratively corrects BC's errors by querying the expert for labels on the agent's own visited states.

Inverse Reinforcement Learning (IRL): Infers the hidden reward function that explains the expert's behavior, rather than just mimicking actions.

Adversarial Imitation Learning (AIL): A discriminator network judges if a trajectory is from the expert or the agent, training the agent to deceive the discriminator.



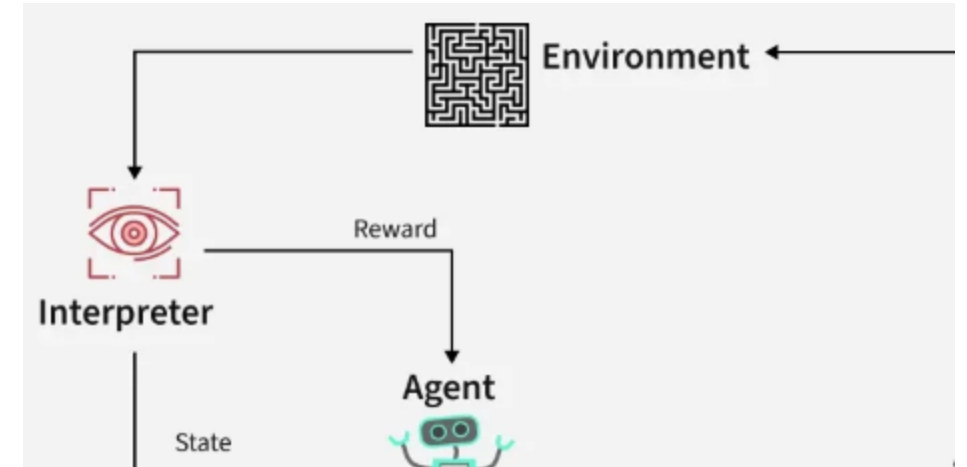
Learning Paradigms for Navigation Models

Reinforcement Learning:

Core Idea: An agent learns optimal actions through trial and error in an environment.

Mechanism: The agent maximizes a cumulative reward signal by discovering which behaviors yield the best long-term outcomes.

Analogy: Learning to navigate by exploration and receiving feedback (rewards/punishments).



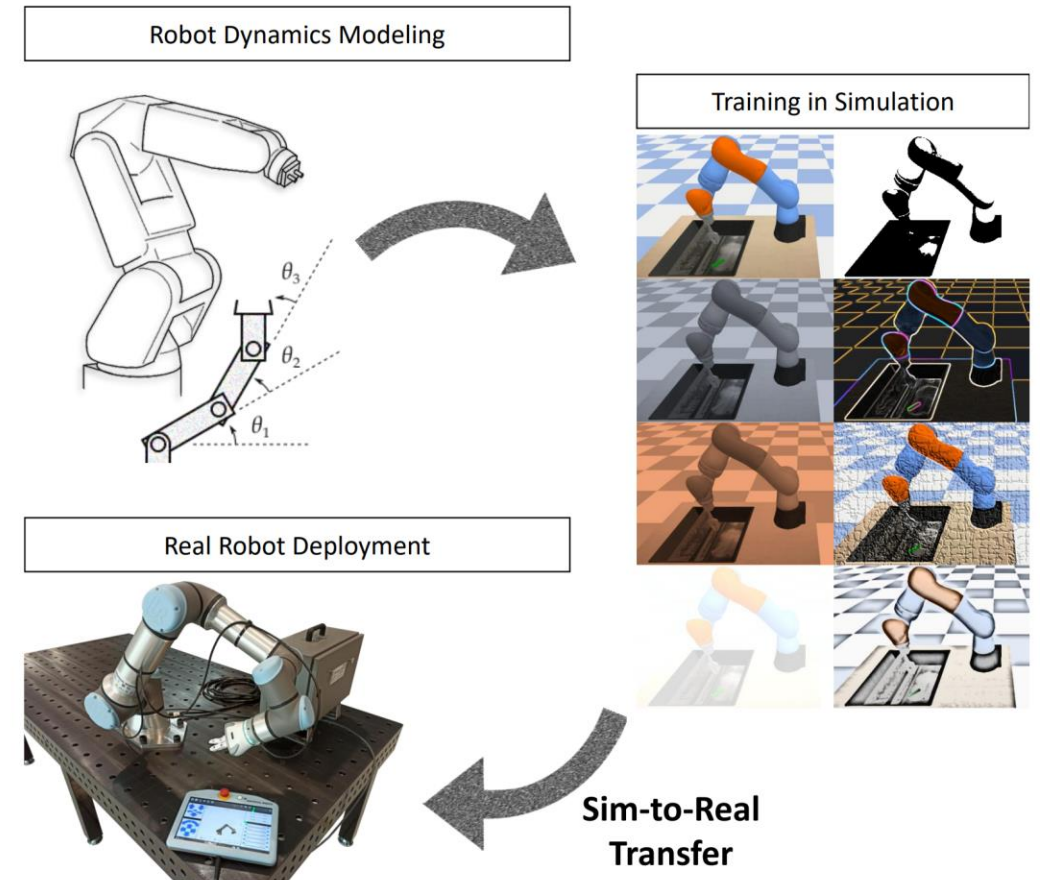
Learning Paradigms for Navigation Models

Transfer Learning

Core Idea: Leverage pre-existing knowledge from a source task to accelerate learning and improve performance on a new, target task.

Benefit: Avoids "starting from scratch," saving significant compute resources and data requirements.

Analogy: A professional racer using their skills to quickly master a new type of vehicle.



New Trends and Discussion

Open Questions

1. The Architectural Dilemma

What is the optimal balance between interpretable modular systems and end-to-end models?

2. The Scalability vs. Efficiency Trade-off

How can we develop end-to-end models that are both large enough to be capable and small enough for real-time deployment?

3. Guaranteeing Safety & Alignment

How do we provably integrate hard safety constraints and ethical rules into data-driven learning systems?

New Trends and Discussion

Key Challenges in Embodied AI

- Sim-to-real transfer
- Long-horizon memory
- 3D reasoning + object permanence
- Social navigation
- Safety + human comfort
- Efficient data collection

Summary

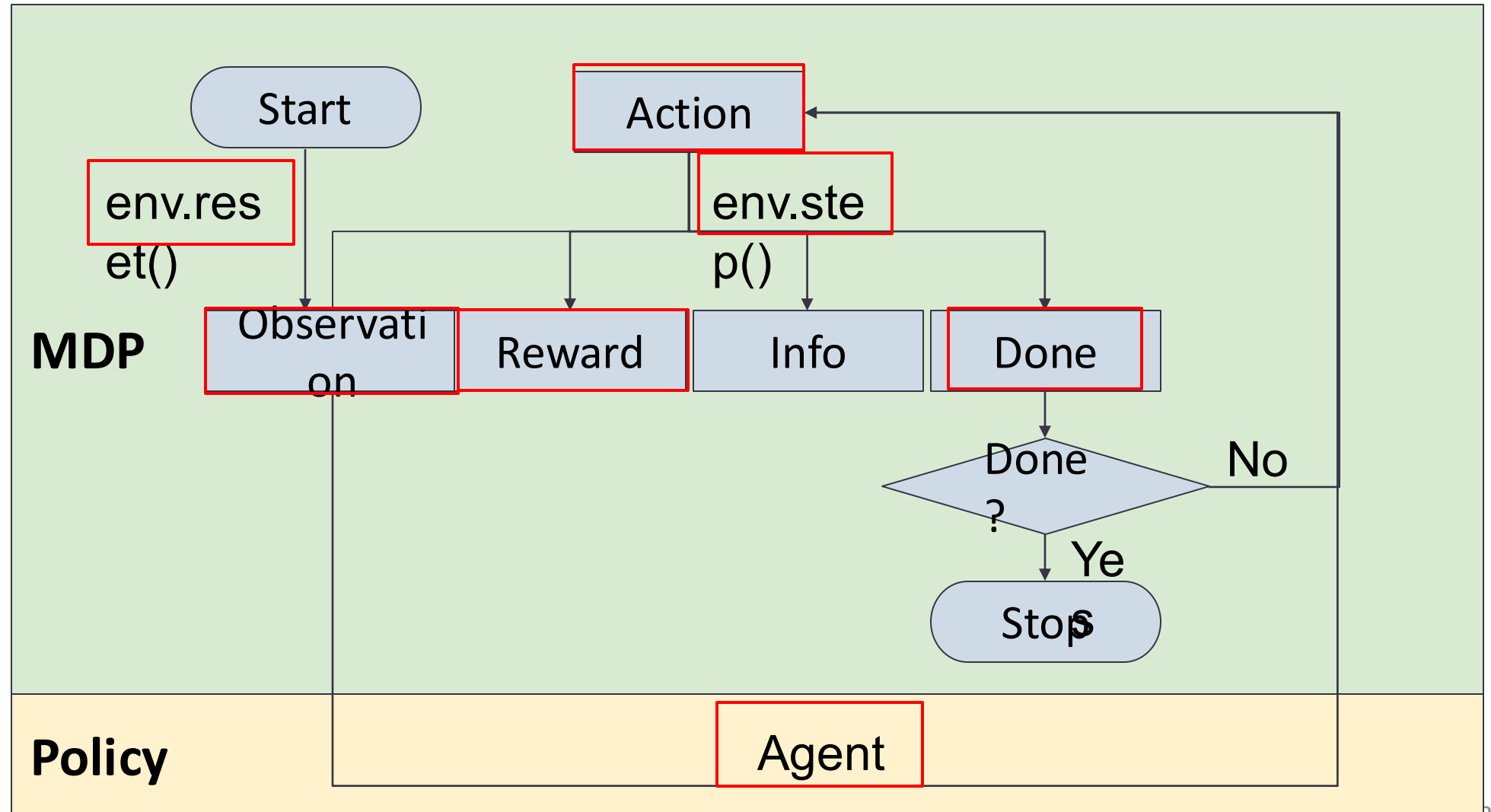
- Self-driving → hybrid E2E stacks w/ BEV transformers
- Embodied AI → multimodal learning (vision-language-action)
- World models & robot foundation models emerging
- Safety, reliability, grounding remain hardest problems

Project 3 Guide

Overview: Simulating MDP on a Computer

The process loop is similar across Isaac Lab/Habitat Lab.

Look for these **key concepts** to understand the code:



Project 3 Guide

Overview: Important Script Locations

*Habitat Lab code is heavily nested.
Isaac Lab is way better

- **The MDP trainer code:** `Falcon/habitat-baselines/habitat_baselines/rl/ppo/falcon_trainer.py` (Do not recommend changing this if you are not an expert)
- **Observations and actions:** `Falcon/habitat-lab/habitat/tasks/rearrange/social_nav_v2/socialnav_task.py` (Do not recommend change this since evaluation fixes the observation/action space)
 - Observations: `Falcon/falcon/additional_sensor.py`
 - Actions: `Falcon/falcon/additional_action.py`
- **The reward function config:** `Falcon/habitat-lab/habitat/config/habitat/task/falcon_task_detail.yaml`
- **The reward function code:** `Falcon/falcon/additional_metric.py`
 - Other default (PointGoal) reward function code: `Falcon/habitat-lab/habitat/tasks/nav/nav.py`
- **The policy and PPO config:** `Falcon/habitat-baselines/habitat_baselines/config/social_nav_v2/falcon_hm3d_train_mini_junwei.yaml`
- **The policy (PointNavResNetPolicy) code:** `Falcon/habitat-baselines/habitat_baselines/rl/ddppo/policy/resnet_policy.py`
- **The original Habitat config guide:** `Falcon/habitat-lab/habitat/config/CONFIG_KEYS.md`

Project 3 Guide

Defining the MDP: falcon_trainer.py

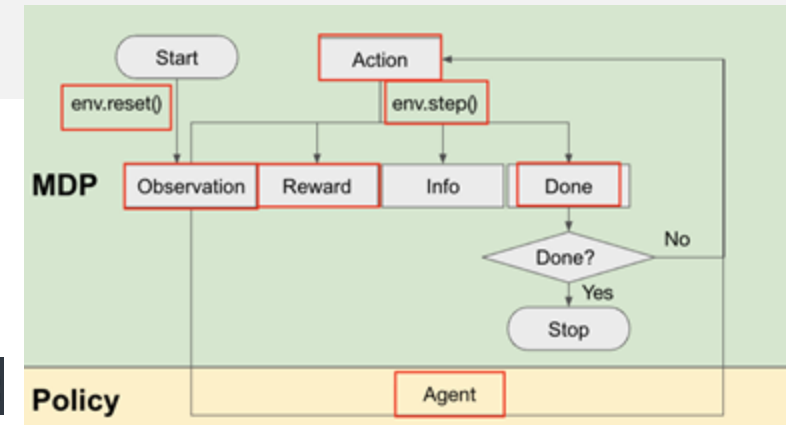
- Under train function:
- The simulation training loop: `while not self.is_done():`
 - Agent code: Falcon/habitat-baselines/habitat_baselines/rl/ppo/single_agent_access_mgr.py
 - Note that are multiple agent instances and we only train agent_0
 - 7 agents, the other 6 are the random walk human agents
 - In each trainer loop, we do two things:
 - **Rollout and collect rewards**
 - **Update agent/policy**

```
profiling_wrapper.range_push("rollouts loop")
profiling_wrapper.range_push(" collect rollout step")
with g_timer.avg_time("trainer.rollout_collect"):
    profiling_wrapper.range_pop() # rollouts loop

if self._is_distributed:
    self.num_rollouts_done_store.add("num_done", 1)

losses = self._update_agent()

self.num_updates_done += 1
```



```
rl:
  agent:
    type: "MultiAgentAccessMgr"
    num_agent_types: 7
    num_active_agents_per_type: [1, 1, 1, 1, 1, 1, 1]
    num_pool_agents_per_type: [1, 1, 1, 1, 1, 1, 1]
    agent_sample_interval: 20
    force_partner_sample_idx: -1
  policy:
    agent_0:
      name: "PointNavResNetPolicy"
```

Rollout MDP: Get observations, current policy outputs actions, step env with the action, [collect rewards], until all done.

Num_steps == maximum step per env for PPO

(ppo.num_steps / habitat.environment.max_episode_steps)

Num_envs == parallel env at the same time

(habitat_baselines.num_environments)

```
for step in range(self._ppo_cfg.num_steps):
```

```
def _compute_actions_and_step_envs(self,
    num_envs = self.envs.num_envs
```

Project 3 Guide

Defining the MDP: falcon_trainer.py

- Under train function:
- The simulation training loop `while not self.is_done():`
 - Agent code: Falcon/habitat-baselines/habitat_baselines/rl/ppo/single_agent_access_mgr.py
 - Note that are multiple agent instances and we only train agent_0
 - 7 agents, the other 6 are the random walk human agents
 - In each trainer loop, we do two things:
 - **Rollout and collect rewards**
 - **Update agent/policy**

```
profiling_wrapper.range_push("rollouts loop")
profiling_wrapper.range_push(" collect rollout step")
with g_timer.avg_time("trainer.rollout_collect"):
profiling_wrapper.range_pop() # rollouts loop

if self._is_distributed:
    self.num_rollouts_done_store.add("num_done", 1)

losses = self._update_agent()

self.num_updates_done += 1
```

Update agent: using PPO, we update the actor and the critic at the same time using gradient descent

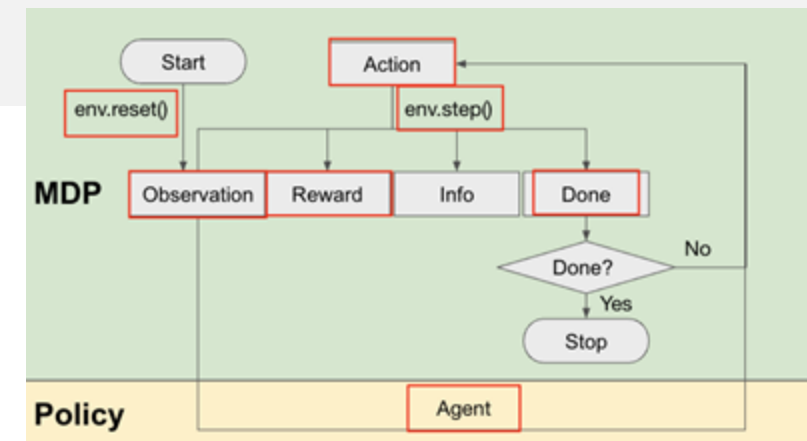
```
rl:
agent:
  type: "MultiAgentAccessMgr"
  num_agent_types: 7
  num_active_agents_per_type: [1, 1, 1, 1, 1, 1, 1]
  num_pool_agents_per_type: [1, 1, 1, 1, 1, 1, 1]
  agent_sample_interval: 20
  force_partner_sample_idx: -1
policy:
  agent_0:
    name: "PointNavResNetPolicy"
```

```
def _update_agent(self):
    with inference_mode():
        self._agent.rollouts.compute_returns(
            next_value,
            self._ppo_cfg.use_gae,
            self._ppo_cfg.gamma,
            self._ppo_cfg.tau,
        )

        self._agent.train()

        losses = self._agent.updater.update(s
        self._agent.rollouts.after_update()
        self._agent.after_update()

    return losses
```



Project 3 Guide

Observations and Actions

*Habitat Lab code is heavily nested.
Isaac Lab is way better

- **The task definition and observations:** Falcon/habitat-lab/habitat/tasks/rearrange/social_nav_v2/socialnav_task.py (Do not recommend change this since evaluation fixes the observation/action space)

```
@add_perf_timing_func()
def _get_observations(self, episode):
    # Fetch the simulator observations, all visual sensors.
    obs = self._sim.get_sensor_observations()

    if not self._sim.sim_config.enable_batch_renderer:
        # Post-process visual sensor observations
        obs = self._sim._sensor_suite.get_observations(obs)
    else:
        # Keyframes are added so that the simulator state can be reconstituted w
        # The post-processing step above is done after batch rendering.
        self._sim.add_keyframe_to_observations(obs)

    # Task sensors (all non-visual sensors)
    obs.update(
        self.sensor_suite.get_observations(
            observations=obs, episode=episode, task=self, should_time=True
        )
    )
    return obs
```

Project 3 Guide

Observations and Actions

*Habitat Lab code is heavily nested.
Isaac Lab is way better

- **Observations:** Falcon/falcon/additional_sensor.py (Do not recommend change this since evaluation fixes the observation/action space)

The auxiliary loss related sensors are defined

```
@registry.register_sensor
class SocialCompassSensor(UsesArticulatedAgentInterface, Sensor):
    """
    Implementation of people relative position sensor
    """

    cls_uuid: str = "social_compass_sensor"

    def __init__(
        self, sim, config, *args, **kwargs
    ):
        self._sim = sim
        # parameters
        self.thres = config.thres
        self.num_bins = config.num_bins
        super().__init__(config=config)

    def _get_uuid(self, *args, **kwargs):
        return self.cls_uuid

    def _get_sensor_type(self, *args, **kwargs):
        return SensorTypes.MEASUREMENT

    def _get_observation_space(self, *args, **kwargs):
        return spaces.Box(low=0, high=np.inf, shape=(self.num_bins,),
                           dtype=np.float32)

    def get_polar_angle(self, agent_id = 0):
        agent_state = self._sim.get_agent_state(agent_id)
        # quaternion is in x, y, z, w format
        ref_rotation = agent_state.rotation
```

Project 3 Guide

Observations and Actions

*Habitat Lab code is heavily nested.
Isaac Lab is way better

- **Actions:** Falcon/falcon/additional_action.py (Do not recommend change this since evaluation fixes the observation/action space)

```
habitat/task/actions@habitat.task.actions.agent_0_discrete_stop: discrete_stop
habitat/task/actions@habitat.task.actions.agent_0_discrete_move_forward: discrete_move_forward
habitat/task/actions@habitat.task.actions.agent_0_discrete_turn_left: discrete_turn_left
habitat/task/actions@habitat.task.actions.agent_0_discrete_turn_right: discrete_turn_right
```

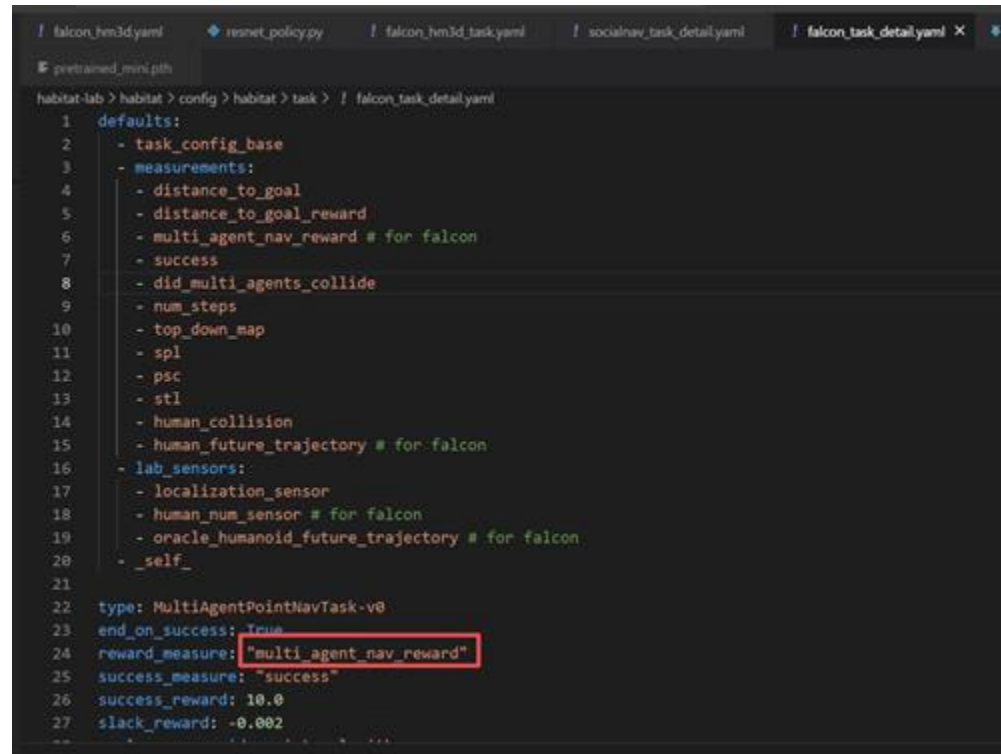
```
@dataclass
class DiscreteMoveForwardActionConfig(ActionConfig):
    type: str = "DiscreteMoveForwardAction"
    lin_speed: float = 10.0
    ang_speed: float = 0.0
    allow_back: bool = False
    value: int = 1
    allow_dyn_slide: bool = False # True
    leg_animation_checkpoint: str = (
        "data/robots/spot_data/spot_walking_trajectory.csv"
    )
    play_i_perframe: int = 5
    use_range: Optional[List[int]] = field(default_factory=lambda: [107, 863])
```


Project 3 Guide

Reward (Try to change this!)

*Habitat Lab code is heavily nested.
Isaac Lab is way better

- **The reward function config:** Falcon/habitat-lab/habitat/config/habitat/task/falcon_task_detail.yaml
- **The reward function code:** Falcon/falcon/additional_metric.py
 - Other default (PointGoal) reward function code: Falcon/habitat-lab/habitat/tasks/nav/nav.py



```
falcon_hm3d.yaml  resnet_policy.py  falcon_hm3d_task.yaml  socialnav_task_detail.yaml  falcon_task_detail.yaml X
pretrained_mini.pth
habitat-lab > habitat > config > habitat > task > falcon_task_detail.yaml
1 defaults:
2   - task_config_base
3   - measurements:
4     - distance_to_goal
5     - distance_to_goal_reward
6     - multi_agent_nav_reward # for falcon
7     - success
8     - did_multi_agents_collide
9     - num_steps
10    - top_down_map
11    - spl
12    - psc
13    - stl
14    - human_collision
15    - human_future_trajectory # for falcon
16  - lab_sensors:
17    - localization_sensor
18    - human_num_sensor # for falcon
19    - oracle_humanoid_future_trajectory # for falcon
20  - _self_
21
22 type: MultiAgentPointNavTask-v0
23 end_on_success: True
24 reward_measure: "multi_agent_nav_reward"
25 success_measure: "success"
26 success_reward: 10.0
27 slack_reward: -0.002
```

Project 3 Guide

Reward (Try to change this!)

*Habitat Lab code is heavily nested.
Isaac Lab is way better

- **The reward function code:** Falcon/falcon/additional_metric.py
 - Other default (PointGoal) reward function code: Falcon/habitat-lab/habitat/tasks/nav/nav.py

```
def update_metric(self, *args, episode, task, observations, **kwargs):  
    # Start social nav reward  
    social_nav_reward = 0.0  
  
    # Component 1: Goal distance reward (strengthened by multiplying by 1.5)  
    distance_to_goal_reward = task.measurements.measures[  
        DistanceToGoalReward.cls_uuid  
    ].get_metric()  
    social_nav_reward += 1.5 * distance_to_goal_reward # Slightly reduced reward  
  
    # Component 2: Penalize being too close to humans  
    distance_to_target = task.measurements.measures[  
        DistanceToGoal.cls_uuid  
    ].get_metric()  
    use_k_robot = f"agent_{self._robot_idx}_localization_sensor"  
    robot_pos = np.array(observations[use_k_robot][:3])  
  
    if distance_to_target > self._allow_distance: ==  
  
    # Component 3: Collision detection for two agents  
    did_agents_collide = task.measurements.measures[  
        DidMultiAgentsCollide._get_uuid()  
    ].get_metric()  
    if did_agents_collide: ==  
  
    # Component 4: Collision detection for the main agent and the scene  
    did_rearrange_collide, collision_detail = rearrange_collision(  
        self._sim, True, ignore_base=False, agent_idx=self._robot_idx  
    )  
    if did_rearrange_collide:  
        social_nav_reward += self._collide_scene_penalty  
  
    # Component 5: Trajectory overlap penalty with time-based weighting  
    if distance_to_target > self._allow_distance and "human_future_trajectory"  
        self._metric = social_nav_reward
```

Component 1: Goal distance reward

Component 2: Penalize being too close to humans

Component 3: Collision detection for two agents

Component 4: Collision detection for the main agent and the scene

Component 5: Trajectory overlap penalty with time-based weighting

Project 3 Guide

Policy (Try to change this!)

*Habitat Lab code is heavily nested.
Isaac Lab is way better

- **The policy and PPO config:** Falcon/habitat-baselines/habitat_baselines/config/social_nav_v2/falcon_hm3d_train_mini_junwei.yaml

```
habitat_baselines:
  evaluate: False # True
  verbose: True
  trainer_name: "falcon_trainer" ## original p
  torch_gpu_id: 0
  tensorboard_dir: "evaluation/falcon/hm3d/tb"
  video_dir: "evaluation/falcon/hm3d/video"
  test_episode_count: -1
  num_environments: 4 # parallel env numbers
  checkpoint_folder: "evaluation/falcon/hm3d/c
  num_updates: -1
  total_num_steps: 800000 # 800K steps
  log_interval: 10
  num_checkpoints: 20 # interrupt recovery
  # Force PyTorch to be single threaded as
  # this improves performance considerably
  force_torch_single_threaded: True
  load_resume_state_config: False
```

```
ppo:
  # ppo params
  clip_param: 0.2
  ppo_epoch: 2
  num_mini_batch: 2
  value_loss_coef: 0.5
  entropy_coef: 0.01
  lr: 1.5e-4
  eps: 1e-5
  max_grad_norm: 0.2
  num_steps: 128
  use_gae: True
  gamma: 0.99
  tau: 0.95
  use_linear_clip_decay: False
  use_linear_lr_decay: False
  reward_window_size: 50

  use_normalized_advantage: False

  hidden_size: 128
```

Do not simply change the config
without understanding what the hyper-
parameter does!

* `assert num_environments >=`
`num_mini_batch`
Usually `num_env == mini_batch`
`total_num_steps`: max training steps

```
# Model parameters
backbone: resnet18
rnn_type: LSTM
num_recurrent_layers: 2
```

Project 3 Guide

Policy (Try to change this!)

- **The policy and PPO config:** Falcon/habitat-baselines/habitat_baselines/config/social_nav_v2/falcon_hm3d_train_mini_junwei.yaml

```
ppo:
  # ppo params
  clip_param: 0.2
  ppo_epoch: 2
  num_mini_batch: 2
  value_loss_coef: 0.5
  entropy_coef: 0.01
  lr: 1.5e-4
  eps: 1e-5
  max_grad_norm: 0.2
  num_steps: 128
  use_gae: True
  gamma: 0.99
  tau: 0.95
  use_linear_clip_decay: False
  use_linear_lr_decay: False
  reward_window_size: 50

  use_normalized_advantage: False

  hidden_size: 128
```

Do not simply change the config without understanding what the hyper-parameter does!

* **ppo_epoch: 2**
update actor/critic 2 times given one rollout

Falcon/habitat-baselines/habitat_baselines/rl/ppo/ppo.py

```
def update(
    self,
    rollouts: RolloutStorage,
) -> Dict[str, float]:
    advantages = self.get_advantages(rollouts)

    learner_metrics: Dict[str, List[Any]] = collections.defaultdict(list)

    for epoch in range(self.ppo_epoch):
        profiling_wrapper.range_push("PPO.update epoch")
        data_generator = rollouts.data_generator(
            advantages, self.num_mini_batch
        )

        for _bid, batch in enumerate(data_generator):
            self._update_from_batch(
                batch, epoch, rollouts, learner_metrics
            )

        profiling_wrapper.range_pop() # PPO.update epoch

    self._set_grads_to_none()
```


Project 3 Guide

Policy (Try to change this!)

*Habitat Lab code is heavily nested.
Isaac Lab is way better

- **The policy and PPO config:** Falcon/habitat-baselines/habitat_baselines/config/social_nav_v2/falcon_hm3d_train_mini_junwei.yaml

```
ppo:
  # ppo params
  clip_param: 0.2
  ppo_epoch: 2
  num_mini_batch: 2
  value_loss_coef: 0.5
  entropy_coef: 0.01
  lr: 1.5e-4
  eps: 1e-5
  max_grad_norm: 0.2
  num_steps: 128
  use_gae: True
  gamma: 0.99
  tau: 0.95
  use_linear_clip_decay: False
  use_linear_lr_decay: False
  reward_window_size: 50

  use_normalized_advantage: False

  hidden_size: 128
```

Do not simply change the config
without understanding what the
hyper-parameter does!

* num_steps: 128
For each rollout, we collect
maximum 128 steps for training

Falcon/habitat-
baselines/habitat_baselines/rl/ppo/falcon_
trainer.py

```
with g_timer.avg_time("trainer.rollout_collect"):
    for buffer_index in range(self._agent.nbuffers):
        self._compute_actions_and_step_envs(buffer_index)

    for step in range(self._ppo_cfg.num_steps):
        is_last_step = (
            self.should_end_early(step + 1)
            or (step + 1) == self._ppo_cfg.num_steps
        )

        for buffer_index in range(self._agent.nbuffers):
            count_steps_delta += (
                self._collect_environment_result(buffer_index)
            )

            if (buffer_index + 1) == self._agent.nbuffers:
                profiling_wrapper.range_pop() # _collect_rollout_

            if not is_last_step:
                if (buffer_index + 1) == self._agent.nbuffers:
                    profiling_wrapper.range_push(
                        "_collect_rollout_step"
                    )

                self._compute_actions_and_step_envs(
                    buffer_index
                )

        if is_last_step:
            break
```


Project 3 Guide

Policy (Try to change this!)

- **The policy (PointNavResNetPolicy) code:** Falcon/habitat-baselines/habitat_baselines/rl/ddppo/policy/resnet_policy.py

*Habitat Lab code is heavily nested.
Isaac Lab is way better

```
# Model parameters
backbone: resnet18
rnn_type: LSTM
num_recurrent_layers: 2
```

Here you can change the
neural network architecture!

```
@baseline_registry.register_policy
class PointNavResNetPolicy(NetPolicy):
    def __init__(
        self,
        observation_space: spaces.Dict,
        action_space,
        hidden_size: int = 512,
        num_recurrent_layers: int = 1,
        rnn_type: str = "GRU",
        resnet_baseplanes: int = 32,
        backbone: str = "resnet18",
        normalize_visual_inputs: bool = False,
        force_blind_policy: bool = False,
        policy_config: "DictConfig" = None,
        aux_loss_config: Optional["DictConfig"] = None,
        fuse_keys: Optional[List[str]] = None,
        **kwargs,
    ):
        """
        Keyword arguments:
        rnn_type: RNN layer type; one of ["GRU", "LSTM"]
        backbone: Visual encoder backbone; one of ["resnet18", "resnet50", "resnet50", "resnet50", "resnet50"]
        """

        assert backbone in [
            "resnet18",
            "resnet50",
            "resnet50",
            "se_resnet50",
            "se_resnet50",
            "se_resnet50",
            "se_resnet50",
            "resnet50_clip_avgpool",
            "resnet50_clip_attnpool",
        ], f"{backbone} backbone is not recognized."
```

Project 3 Guide

More Config Explanation from the Original Habitat Lab

*Habitat Lab code is heavily nested.
Isaac Lab is way better

- **The original Habitat config guide:** Falcon/habitat-lab/habitat/config/CONFIG_KEYS.md

If you want to learn more about Habitat Lab Tasks Configuration...

Configuration Keys

Dataset

Configuration for the dataset of the task. A dataset consists of episodes (a start configuration for a task within a scene) and a scene dataset (with all the assets needed to instantiate the task). For a list of all the available datasets, see [this page](#).

Key	Description
habitat.dataset.type	The key for the dataset class that will be used. Examples of such keys are <code>PointNav-v1</code> , <code>ObjectNav-v1</code> , <code>InstanceImageNav-v1</code> or <code>RearrangeDataset-v0</code> . Different datasets have different properties so you should use the dataset that fits your task.
habitat.dataset.scene_dir	The path to the directory containing the scenes that will be used. You should put all your scenes in the same folder (example <code>data/scene_datasets</code>) to avoid having to change it.
habitat.dataset.data_path	The path to the episode dataset. Episodes need to be compatible with the <code>type</code> argument (so they will load properly) and only use scenes that are present in the <code>scenes_dir</code> .
habitat.dataset.split	<code>data_path</code> can have a <code>split</code> in the path. For example: <code>"data/datasets/pointnav/habitat-test-scenes/v1/{split}/{split}.json.gz"</code> the value in <code>"{split}"</code> will be replaced by the value of the <code>split</code> argument. This allows to easily swap between training, validation and test episodes by only changing the <code>split</code> argument.

Task



Thanks for your attention!

Changhao Chen
HKUST (GZ)

changhaochen@hkust-gz.edu.cn

Homepage: [changhao-chen@github.io](https://github.com/changhao-chen)